

Đề Tài Đồ Án Môn Học Về Thuật Toán và Ứng Dụng Học Máy

PHẦN I: HỌC MÁY CÓ GIÁM SÁT (SUPERVISED LEARNING)

Các đề tài này dựa trên các thuật toán Học máy có giám sát được thảo luận trong Mục I của tài liệu.

Đề tài 1: Triển khai Cây Quyết Định (Decision Tree) C4.5 từ đầu (Scratch) trong Python

- **Mô tả:** Nghiên cứu sâu về khái niệm **Entropy** và **Information Gain** (là cơ sở của thuật toán ID3), sau đó tập trung triển khai thuật toán **C4.5** sử dụng độ đo **Gain Ratio** để giải quyết vấn đề thiên vị (bias problem) của Information Gain, áp dụng cho một tập dữ liệu phân loại.
- **Yêu cầu Python:** Xây dựng lớp (class) DecisionTree bằng Python, tính toán Entropy, Information Gain, và Gain Ratio bằng thư viện NumPy.

Đề tài 2: Phân loại Naïve Bayesian và kỹ thuật ước lượng Laplace (Laplace Estimator)

- **Mô tả:** Áp dụng Lý thuyết xác suất Bayes' Theorem. Triển khai bộ phân loại Naïve Bayesian để phân loại dữ liệu văn bản (Text Classification) hoặc dữ liệu có thuộc tính rời rạc. Tập trung giải quyết vấn đề **xác suất bằng không (zero probability)** bằng cách sử dụng **Laplace Estimator** (hay còn gọi là Laplace Correction).
- **Yêu cầu Python:** Sử dụng thư viện Scikit-learn (hoặc triển khai thủ công) để xây dựng mô hình, đồng thời chứng minh và áp dụng hiệu quả của Laplace Estimator trong Python.

Đề tài 3: Xây dựng Bộ phân loại k-Nearest Neighbors (k-NN) và Phân tích Trọng số Khoảng cách

- **Mô tả:** Triển khai bộ phân loại k-NN, một phương pháp học lười (lazy learning). So sánh hiệu suất phân loại khi sử dụng các phương pháp trọng số (weighting schemes) khác nhau cho hàng xóm (ví dụ: trọng số bằng nhau hoặc **trọng số nghịch đảo khoảng cách (Inverse Weights)**) trên một tập dữ liệu chuẩn như Iris hoặc Ecoli.
- **Yêu cầu Python:** Sử dụng thư viện Scikit-learn để tinh chỉnh tham số k và thử nghiệm các hàm khoảng cách (distance metrics) khác nhau, cùng với việc áp dụng trọng số.

Đề tài 4: Phân loại phi tuyến tính bằng Support Vector Machine (SVM) và Kernel Trick

- **Mô tả:** Nghiên cứu nguyên lý hoạt động của SVM nhằm tìm ra siêu mặt phẳng (hyperplane) tối ưu hóa khoảng cách (margin) giữa các Support Vectors. Áp dụng **Kernel Trick** để xử lý dữ liệu không thể phân tách tuyến tính (nonlinearly separable data), tập trung sử dụng và đánh giá hiệu suất của **Hàm Kernel Gaussian (Radial Basis Function - RBF)**.
- **Yêu cầu Python:** Sử dụng thư viện Scikit-learn's SVC, tập trung vào việc điều chỉnh tham số C và tham số γ (gamma) của Kernel RBF để tối ưu hóa hiệu suất.

Đề tài 5: Xây dựng và Huấn luyện Mạng Neural Perceptron Đa Lớp (MLP) với Backpropagation

- **Mô tả:** Phân tích sự hạn chế của Perceptron đơn giản và xây dựng mô hình **Multilayer Perceptron Network** để giải quyết các bài toán phi tuyến (như XOR problem). Triển khai thuật toán **Backpropagation** để điều chỉnh trọng số (weights updates) thông qua việc sử dụng hàm kích hoạt **Sigmoid**, vì Backpropagation yêu cầu hàm khả vi (differentiable).
- **Yêu cầu Python:** Sử dụng thư viện Keras/TensorFlow hoặc Scikit-learn để xây dựng và huấn luyện mô hình MLP, giám sát hiệu suất (performance) và lỗi (error) qua các epoch.

Đề tài 6: Phân tích Quy trình Xây dựng và Cắt tỉa Luật bằng Thuật toán Ripper (JRip)

- **Mô tả:** Nghiên cứu về Rule-Based Classifiers và Thuật toán **Ripper** (Repeated Incremental Pruning). Phân tích chi tiết các giai đoạn: **Building Stage** (Growing Rule - phát triển luật), **Prune Phase** (cắt tỉa luật để chống overfitting), và **Optimization Stage** (tối ưu hóa luật).
- **Yêu cầu Python:** Sử dụng thư viện Python như `Weka-Wrapper` (nếu có thể tích hợp JRip) hoặc triển khai một phiên bản đơn giản của thuật toán Sequential Covering Algorithm để minh họa quy trình xây dựng và cắt tỉa luật.

PHẦN II: HỌC MÁY KHÔNG GIÁM SÁT VÀ ỨNG DỤNG (UNSUPERVISED LEARNING & APPLICATIONS)

Các đề tài này dựa trên các thuật toán Học máy không giám sát được thảo luận trong Mục II của tài liệu, bao gồm K-means, GMM, HMM, và PCA.

Đề tài 7: Phân tích Thuật toán K-Means Clustering và Nearest Centroid Classifier

- **Mô tả:** Triển khai thuật toán **K-Means Clustering** (dạng lượng tử hóa vector) để phân vùng dữ liệu không nhãn. Phân tích quy trình lặp để tối thiểu hóa hàm chi phí (J). Sau khi tìm được tâm cụm (cluster centers), sử dụng các tâm cụm này để xây dựng mô hình **Nearest Centroid Classifier (Rocchio algorithm)** nhằm phân loại các điểm dữ liệu mới.
- **Yêu cầu Python:** Sử dụng Scikit-learn's `KMeans` để gom cụm và Scikit-learn's `NearestCentroid` để phân loại, so sánh hiệu suất khi thay đổi giá trị K .

Đề tài 8: Gom cụm bằng Mô hình Hỗn hợp Gaussian (GMM) và Thuật toán Expectation Maximization (EM)

- **Mô tả:** Nghiên cứu Mô hình **Gaussian Mixture Model (GMM)**, bao gồm các tham số mô hình (trọng số w_k , mean μ_k , covariance Σ_k). Áp dụng thuật toán Lặp **Expectation Maximization (EM)** để tìm ra các tham số mô hình phù hợp nhất với dữ liệu, nhằm gán xác suất hậu nghiệm (posterior probability) cho các điểm dữ liệu vào các cụm tương ứng.

- **Yêu cầu Python:** Sử dụng thư viện Scikit-learn's `GaussianMixture` để triển khai, trực quan hóa các đường viền của các cụm Gaussian.

Đề tài 9: Giảm chiều dữ liệu bằng Principal Component Analysis (PCA)

- **Mô tả:** Nghiên cứu cách **PCA** sử dụng phân tích ma trận hiệp phương sai (covariance matrix) và **SVD (Singular Value Decomposition)** để tìm ra các thành phần chính (Principal Components). Xác định số lượng thành phần chính cần thiết để giữ lại tỷ lệ phương sai (variance retained) mong muốn. Đánh giá **Lỗi tái tạo dữ liệu (Data Reconstruction Error)**.
- **Yêu cầu Python:** Sử dụng Scikit-learn's `PCA` để thực hiện giảm chiều, trực quan hóa dữ liệu 2D đã giảm chiều, và tính toán lỗi tái tạo.

Đề tài 10: Ứng dụng Mô hình Markov Ẩn (HMM) trong Nhận dạng Chuỗi (Sequence Recognition)

- **Mô tả:** Nghiên cứu cấu trúc **Hidden Markov Model (HMM)**, bao gồm các trạng thái ẩn (invisible/unobservable) và các vấn đề cơ bản của HMM. Tập trung vào **Bài toán Căn chỉnh (Alignment problem)**, ví dụ như suy luận chuỗi trạng thái ẩn (ví dụ: thái độ của sếp) từ chuỗi quan sát được.
- **Yêu cầu Python:** Sử dụng thư viện Python như `hmmlearn` để xây dựng và huấn luyện một mô hình HMM đơn giản, sau đó sử dụng thuật toán Viterbi để suy luận chuỗi trạng thái ẩn tối ưu.

Đề tài 11: Phát hiện Sâu máy tính đa hình (Polymorphic Worms) bằng PCA Điều chỉnh (MPCA)

- **Mô tả:** Nghiên cứu bài toán phát hiện sâu đa hình bằng thuật toán **PCA Điều chỉnh (MPCA)** (kết hợp với SEA - Substring Exaction Algorithm) và thuật toán **MKMP (Modified Knuth–Morris–Pratt)**. Phân tích quy trình trích xuất chuỗi con (substring extraction) và sử dụng PCA để xác định **đặc trưng quan trọng nhất (most significant data)** của sâu.
- **Yêu cầu Python:** Mô phỏng quy trình của MPCA: Tạo ma trận tần suất (Frequency Matrix) từ các chuỗi con mẫu, chuẩn hóa, và sử dụng thư viện PCA (hoặc NumPy SVD) để trích xuất các thành phần chính đại diện cho "chữ ký" của sâu.

Đề tài 12: So sánh Hiệu suất của các Bộ phân loại Học máy trong Chẩn đoán Y khoa Hỗ trợ bằng Máy tính (CAD)

- **Mô tả:** Nghiên cứu ứng dụng của Học máy trong **Chẩn đoán Y khoa Hỗ trợ bằng Máy tính (CAD)**. So sánh hiệu suất (performance) của ít nhất ba bộ phân loại Học máy có giám sát (ví dụ: Naïve Bayesian, Neural Network, và SVM) khi áp dụng để phân tích các tập dữ liệu chẩn đoán bệnh lý (ví dụ: phát hiện ung thư, phân tích hình ảnh y khoa).
- **Yêu cầu Python:** Sử dụng thư viện Scikit-learn để triển khai các mô hình, sử dụng các độ đo đánh giá tiêu chuẩn như Accuracy, Precision, Recall, và F1-Score để so sánh kết quả.